System zgłoszeń w architekturze mikroserwisowej

Projekt w C# do otwarcia i uruchomienia w Visual Studio

ASP.NET Core 8 • Razor Pages • JWT • SQLite • HTTP API Gateway

Status dokumentu

To jest opis przygotowanego projektu, a nie raport z potwierdzonego uruchomienia. W środowisku przygotowania paczki nie było SDK .NET. Nie wykonano kompilacji, testów integracyjnych ani rzeczywistych zrzutów ekranu. Pliki i konfiguracje sprawdzono statycznie.

Przed oddaniem zadania: uruchom rozwiązanie w Visual Studio, wykonaj testy i skrypt Generate-Report.ps1. Narzędzie w C# utworzy PDF „Raport_z_dzialania.pdf” z rzeczywistymi zrzutami. Sam ten dokument nie spełnia jeszcze części wymagania dotyczącej zrzutów.

1. Cel projektu

Realizacja zadania z załączonego PDF-a: system z trzema mikroserwisami, centralnym API Gateway i klientem umożliwiającym rejestrację, logowanie, autoryzowany CRUD oraz podgląd logów. Domeną aplikacji są zgłoszenia, np. problemy z dostępem do raportów.

2. Zgodność z wymaganiami

Wymaganie	Element rozwiązania
Mikroserwis aplikacyjny: CRUD i własna baza	TicketService; tickets.db; utworzenie, lista, odczyt pojedynczy, edycja, usunięcie.
Rejestracja, logowanie, generowanie i weryfikacja JWT	AuthService; auth.db; PasswordHasher; /api/auth/validate.
Rejestrowanie i odczyt logów	LogService; logs.db; audyt działań i odczyt własnych wpisów.
Centralny punkt komunikacji i routing	ApiGateway; wywołania klienta i zapis audytu serwisów przez Gateway.
Klient umożliwiający demonstrację	WebClient, Razor Pages; formularze i wywołania HTTP obsługiwane w C#.
PDF: opis i rzeczywiste zrzuty	Opis: ten dokument. Zrzuty i finalny PDF: po uruchomieniu generatora.

3. Architektura i odpowiedzialności

```
Przeglądarka → WebClient :5200
                    ↓
               ApiGateway :5100
                 ├─ AuthService :5101   → auth.db
                 ├─ TicketService :5102 → tickets.db
                 └─ LogService :5103    → logs.db

AuthService / TicketService
    → Gateway /internal/logs
        → LogService /api/logs
```

Każdy mikroserwis jest osobnym procesem ASP.NET Core i ma własny plik SQLite. Serwisy nie czytają wzajemnie swoich baz. Powiązanie właściciela zgłoszenia to identyfikator użytkownika z claim sub w JWT, a nie klucz obcy do innej bazy.

Projekt	Port	Odpowiedzialność
AuthService	5101	Rejestracja, skróty haseł, uwierzytelnianie i wydawanie JWT.
TicketService	5102	Autoryzowany CRUD własnych zgłoszeń.
LogService	5103	Trwały zapis audytu i odczyt własnych logów.
ApiGateway	5100	Routing HttpClient, JWT, obsługa niedostępności serwisów.
WebClient	5200	Widoki Razor Pages i obsługa formularzy w C#.
Shared	—	Biblioteka kontraktów i infrastruktury; nie mikroserwis.
tools/ReportGenerator	—	Opcjonalny program C# do zrzutów i finalnego PDF-a.

WebClient zna wyłącznie adres Gateway. Gateway weryfikuje JWT przed skierowaniem żądania do chronionego API; serwis docelowy dodatkowo weryfikuje token. AuthService i TicketService wysyłają logi przez /internal/logs z kluczem X-Internal-Key.

4. Przepływ podstawowej operacji

- Użytkownik rejestruje konto; AuthService zapisuje skrót hasła oraz log REGISTER.
- Logowanie zwraca JWT ważny domyślnie 60 minut oraz zapisuje LOGIN.
- Klient przechowuje token w sesji serwera, nie w localStorage.
- Formularz zgłoszenia wywołuje Gateway z nagłówkiem Authorization: Bearer.
- Gateway i TicketService weryfikują JWT. TicketService zapisuje rekord właściciela.
- Log CREATE trafia przez Gateway do LogService. Klient odświeża listę i logi.

5. Model danych

AuthService — tabela Users

Id (GUID jako TEXT), Email (unikalny), PasswordHash, CreatedAtUtc. E-mail jest trimowany i normalizowany do małych liter. Hasło: 8–128 znaków; przechowywany jest wynik PasswordHasher, nie jawne hasło.

TicketService — tabela Tickets

Id (INTEGER AUTOINCREMENT), OwnerId, Title, Description, Status, CreatedAtUtc, UpdatedAtUtc. Tytuł: 3–100 znaków; opis: maksymalnie 2000. Status: Nowe, W toku, Zamknięte. Każdy odczyt, UPDATE i DELETE filtruje OwnerId.

LogService — tabela Logs

Id, TimestampUtc, UserId, Service, Action, Details. Czas UTC jest generowany przez LogService. Panel wyświetla do 100 ostatnich własnych wpisów; API pozwala ustawić limit do 200.

Rejestrowane akcje: REGISTER, LOGIN, LOGIN_FAILED, CREATE, READ, READ_LIST, UPDATE i DELETE. Nieudane logowanie na nieistniejące konto jest zapisywane z pustym UserId; wpis nie jest widoczny w zwykłym panelu użytkownika.

6. Publiczne API Gateway

Metoda i ścieżka	JWT	Wynik
POST /api/auth/register	Nie	201; błędne dane 400; duplikat 409.
POST /api/auth/login	Nie	200 z JWT; błędne dane 400; złe hasło 401.
GET /api/auth/validate	Tak	200 + valid=true; nieważny token 401.
GET /api/tickets	Tak	200 + lista własnych zgłoszeń.
GET /api/tickets/{id}	Tak	200; cudzy lub nieistniejący rekord 404.
POST /api/tickets	Tak	201 + utworzony rekord.
PUT /api/tickets/{id}	Tak	200 + zmieniony rekord; 404.
DELETE /api/tickets/{id}	Tak	204; 404.
GET /api/logs?take=100	Tak	200 + własne wpisy.

Każda aplikacja udostępnia GET /health. Niedostępny serwis docelowy daje w Gateway 502. POST /internal/logs jest ścieżką wewnętrzną wymagającą klucza. Publiczny POST /api/logs nie jest wystawiony przez Gateway.

Przykładowe dane zgłoszenia

```
{
  "title": "Brak dostępu do raportu",
  "description": "Proszę sprawdzić uprawnienia.",
  "status": "Nowe"
}
```

7. Uruchomienie w Visual Studio

1. Zainstaluj Visual Studio 2022 z aktualizacjami, komponent „ASP.NET i tworzenie aplikacji internetowych” oraz SDK .NET 8.

2. Wypakuj całą paczkę i otwórz MicroservicesAssignment.sln. Przywróć pakiety NuGet.

3. Wykonaj Build → Build Solution.

4. We właściwościach rozwiązania wybierz Multiple startup projects / konfigurację wielu projektów startowych.

5. Ustaw Start dla LogService, AuthService, TicketService, ApiGateway i WebClient. Shared pozostaw jako None.

6. Użyj profili o nazwach zgodnych z projektami; nie IIS Express. Uruchom F5 lub Ctrl+F5.

7. Otwórz http://127.0.0.1:5200. Bazy tworzą się automatycznie w Data każdego mikroserwisu.

Wolne muszą być porty 5100–5103 i 5200. Całe rozwiązanie używa lokalnego HTTP. Nie jest wymagany SQL Server, Node.js ani npm.

Alternatywne uruchomienie z PowerShell

```
powershell -ExecutionPolicy Bypass -File
  .\scripts\Start-All.ps1
```

Polecenie wpisz w jednym wierszu z katalogu rozwiązania. Nie uruchamiaj drugiej kopii usług, gdy są już uruchomione przez Visual Studio.

8. Testy API i demonstracja

Przy działających usługach uruchom w osobnej konsoli:

```
powershell -ExecutionPolicy Bypass -File
  .\scripts\Smoke-Test.ps1
```

Skrypt testuje health, brak i uszkodzenie JWT, zabezpieczenie zapisu logów, rejestrację, duplikat e-mail, niepoprawne hasło, walidację danych, pełny CRUD, izolację dwóch kont oraz komplet działań w logach. W razie rozbieżności kończy się błędem. Konta testowe pozostają w bazie; testowe zgłoszenie jest usuwane.

Ręczny scenariusz demonstracyjny

• Zarejestruj konto, np. student@example.com / DemoPass123!; następnie zaloguj się.

• Kliknij „Sprawdź JWT”. Dodaj zgłoszenie i odczytaj listę.

• Kliknij „Edytuj”, zmień tytuł i status, następnie zapisz.

• Usuń zgłoszenie. Pokaż logi CREATE, READ, READ_LIST, UPDATE i DELETE.

• Wyloguj się. Opcjonalnie utwórz drugie konto i sprawdź izolację danych.

9. Finalny PDF ze zrzutami — narzędzie C#

Do paczki dołączono program tools/ReportGenerator oparty na Playwright .NET. Działa na lokalnie uruchomionym WebClient, zakłada osobne losowe konto i wykonuje scenariusz interfejsu. Każdy etap sprawdza oczekiwany komunikat; dopiero zakończony scenariusz tworzy finalny dokument.

```
powershell -ExecutionPolicy Bypass -File
  .\scripts\Generate-Report.ps1
```

Polecenie wpisz w jednym wierszu, po uruchomieniu wszystkich pięciu aplikacji. Pierwsze wykonanie wymaga internetu do pobrania pakietu Playwright i Chromium. Python nie jest wymagany.

Plik PNG	Pokazany etap
01-start.png	Formularze rejestracji i logowania.
02-register.png	Potwierdzenie utworzenia konta.
03-login.png	Aktywna sesja i panel użytkownika.
04-create.png	Dodane zgłoszenie oraz lista i logi.
05-update.png	Zmieniony tytuł i status.
06-jwt.png	Potwierdzenie weryfikacji JWT.
07-delete.png	Usunięcie rekordu i pozostały audyt.
08-logout.png	Powrót do formularzy po wylogowaniu.

Rezultaty po udanym uruchomieniu: PNG w Screenshots, HTML oraz PDF w Documentation. Finalny dokument do oddania: **Raport_z_dzialania.pdf**. Zawiera opis architektury, API, bezpieczeństwa oraz osiem rzeczywistych zrzutów. Ten plik nie istnieje jeszcze w przygotowanej paczce, ponieważ aplikacja nie została tutaj uruchomiona.

Jeżeli scenariusz się nie powiedzie, generator zgłosi błąd. Stary raport z wcześniejszej udanej próby może pozostać na dysku; zawsze sprawdź datę dokumentu i wynik konsoli. Testy Smoke-Test.ps1 są osobną weryfikacją i nie należy utożsamiać ich z samą demonstracją UI.

10. Lista przed oddaniem

☐ Projekt skompilowany w Visual Studio.

☐ Wszystkie pięć aplikacji działa i testy API przechodzą.

☐ Sprawdzono demonstrację interfejsu.

☐ Załączono Raport_z_dzialania.pdf z rzeczywistymi zrzutami.

☐ Usunięto bin, obj, .vs oraz prawdziwe dane i rzeczywiste sekrety.

11. Bezpieczeństwo i ograniczenia

• Konfiguracja zawiera demonstracyjne sekrety. Przed innym użyciem zmień Jwt:Key i InternalKey, najlepiej przez User Secrets lub zmienne środowiskowe. W czterech serwisach ustaw te same parametry JWT i klucz wewnętrzny.

• Walidowane są podpis HS256, issuer, audience i exp. Tolerancja zegara wynosi 10 sekund. Gateway i serwisy sprawdzają JWT niezależnie.

• SQL jest parametryzowany. Widoki Razor kodują tekst. Formularze POST używają ochrony anty-CSRF Razor Pages.

• Token znajduje się w pamięci sesji WebClient. Restart klienta wylogowuje użytkownika. Wylogowanie nie unieważnia tokenu globalnie; brak blacklisty JWT.

• Adresy są ograniczone do loopback. Sam JWT nie blokuje bezpośrednich żądań do serwisów. W innym wdrożeniu ogranicz dostęp sieciowy, użyj TLS i bezpiecznych ciasteczek.

• Logi są best-effort. AuditClient czeka do 3 sekund; awaria audytu nie cofa CRUD lub logowania, a ostrzeżenie trafia do konsoli. Brak outbox i kolejki, więc nie ma gwarantowanego dostarczenia audytu.

• Nie ma ról administratora, MFA, refresh tokenów ani rate limiting. Użytkownik przegląda własne logi.

• Routing HttpClient jest celowo prosty. Nie implementuje pełnego reverse proxy, load balancing, circuit breaker ani WebSocket.

• Pakiety mają zakresy wersji 8.0.* i 1.*. Po udanym przywróceniu warto zamrozić wersje w projekcie przeznaczonym do oddania.

12. Szybka diagnostyka

Objaw	Sprawdzenie
Brak SDK / błąd kompilacji	dotnet --list-sdks; SDK .NET 8 i przywrócone pakiety.
Port zajęty	Zatrzymaj drugą kopię usług lub poprzednie konsole.
502 z Gateway	Uruchom serwis docelowy i sprawdź Services w konfiguracji Gateway.
401 po zmianie konfiguracji	Jednakowe ustawienia JWT; restart usług i nowe logowanie.
Brak zrzutów lub PDF	Uruchom cały system, sprawdź testy API i instalację Chromium.

Dokumentacja technologii: Microsoft Learn — Configure JWT bearer authentication in ASP.NET Core; Microsoft.Data.Sqlite overview. Pełne adresy oraz instrukcje znajdują się w README.md. Podstawą zakresu zadania jest dołączony oryginalny PDF.